ARRAYS
STRINGS
HASHING
TWO POINTERS
SLIDING WINDOW
STACK
QUEUE
LINKED LIST
RECURSION
TREES
GRAPHS
HEAPS
THINK
SOLVE
LEARN
IMPROVE
LeetCode
VOLUME 1
One Problem at a Time
Better Logic Stronger You
Consider Edge Cases
LeetCode
Run
Accepted
Time
0 ms
Memory
7.2 MB
DATA STRUCTURES
ALGORITHMS
PROBLEM SOLVING
PATTERNS
PRACTICE
SAME PROBLEMS STRONGER MINDSET
Approach:
- Understand
- Plan
- Implement
- Dry Run
- Optimize
Time: O(n)
Space: O(n)

2487. Remove Nodes From Linked List (Medium) (Linked List , Reverse Algorithm)

You are given the head of a linked list . Remove every node which has a node with a greater value anywhere to the right side of it . Return the head of the modified linked list.

Example 1:-

head = [5,2,13,3,8]

output = [13,8]

- 13 right of 5 & 13>5
- 13 right of 2 & 13>2
- 8 right of 3 & 8>3

Example 2:-

head = [1,1,1,1]

Output = [1,1,1,1]

- Every node has same value

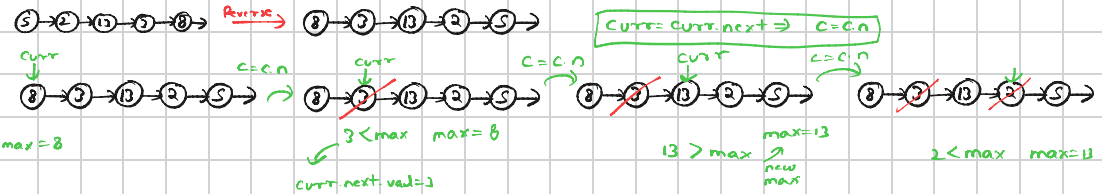
Code:-

```
public ListNode reverseList (ListNode head){
    if (head == null || head.next == null) return head;
    ListNode prev = null;
    ListNode curr = head;

    while (curr != null){
        ListNode temp = curr.next;
        curr.next = prev;
        prev = curr;
        curr = temp;
    }
    return prev;
}
```

This reverse function is already a Leetcode question so not gonna explain the visualisation

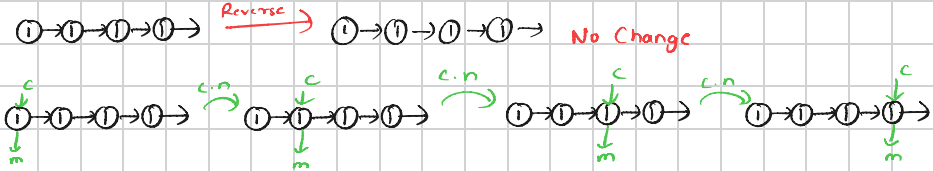
```
public ListNode removeNodes (ListNode head){
    if (head == null || head.next == null) return head;
    head = reverseList(head);
    ListNode curr = head;
    int max = curr.val;
    while (curr.next != null){
        if (curr.next.val < max){
            curr.next = curr.next.next;
        }else{
            curr = curr.next;
            max = curr.val;
        }
    }
    head = reverseList(head);
    return head;
}
```

1) head = [5, 2, 13, 3, 8]

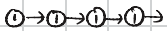


final

2) head = [1, 1, 1, 1]



final


No change.

(2)

75. Sort Colors (Medium) (Arrays, Sorting)

You are given an array nums with n objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white, and blue. We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively. You must solve this problem without using the library's sort function.

Example 1:
Input: nums = [2,0,2,1,1,0]
Output: [0,0,1,1,2,2]

Explanation:
The array has two 0s, two 1s, and two 2s. Sorting them in-place places all 0s first, then all 1s, then all 2s.

Example 2:
Input: nums = [2,0,1]
Output: [0,1,2]

Explanation:
The array has one each of 0, 1, and 2, arranged in-place in the order 0, 1, 2.

Code:-

```
public void SortColors(int[] nums){
    if (nums == null || nums.length <= 1) return;
```

```
for (int i=1; i< nums.length ; i++){
    int key = nums[i];
    int j= i-1;

    while ( j>=0 && nums[j] > key){
        nums[j+1] = nums[j];
        j--;
    }
    nums [j+1]= key;
}
}
```

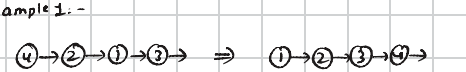
No need for visual explanation as its a direct implementation of a sorting algorithm. And no need for sorting example explaination too

2

147. Insertion Sort List (Medium) (Arrays, LinkedList, Sorting)

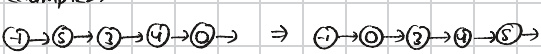
Given the head of a singly linked list, sort the list using insertion sort, and return the sorted list's head.

Example 1:-



head = [4,2,1,3] Output = [1,2,3,4]

Example 2:-



head = [-1,5,3,4,0] Output = [-1,0,3,4,5]

Code:-

```
public int[] linkedListToArray(ListNode head){
    int size =0;
    ListNode curr = head;

    while (curr != null){
        size++;
        curr = curr.next;
    }
    int[] arr = new int[size];
    curr=head;
    int i=0;

    while (curr != null){
        arr[i] = curr.val;
        i++;
        curr. next;
    }
    return arr;
}
```

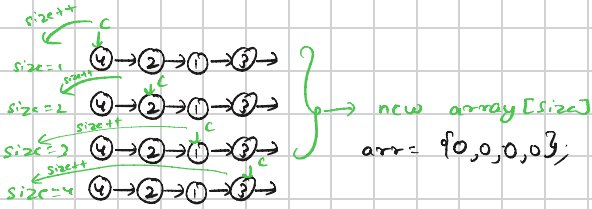
```
public ListNode arrayToLinkedList(int[] arr){
    if (arr ==null || arr.length==0) return null;

    ListNode head = new ListNode(arr[0]);
    ListNode curr= head;

    for(int i=1; i< arr.length; i++){
        curr.next = new ListNode(arr[i]);
        curr = curr.next;
    }
    return head;
}
```

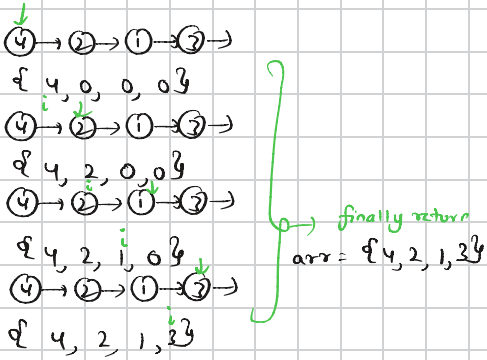
These two function needs visualisation

```
public void insertSort (int [] arr){
    if (arr==null || arr.length ==1) return;
    for (int i=1; i< arr.length; i++){
        int key = arr[i];
        int j= i-1;

        while (j>=0 && arr[j] > key){
            arr[j+1]= arr[j];
            j--;
        }
        arr[++j] = key;
    }
}
```

```
public ListNode insertionSortList (ListNode head){
    int[] arr = linkedListToArray(head);
    insertSort(arr);
    ListNode list = arrayToLinkedList(arr);
    return list;
}
```

① LL → array





② array → LL

{4, 2, 1, 3}

head

4 → {4, 2, 1, 3}

4 → 2 → {4, 2, 1, 3}

4 → 2 → 1 → {4, 2, 1, 3}

4 → 2 → 1 → 3 → } → Return

Eg → 1) head = [4, 2, 1, 3]

① 4 → 2 → 1 → 3 → ⇒ {4, 2, 1, 3}

② {4, 2, 1, 3} ⇒ {1, 2, 3, 4}

③ {1, 2, 3, 4} ⇒ 1 → 2 → 3 → 4 →

Eg → 2) head = [-1, 5, 3, 4, 0]

① -1 → 5 → 3 → 4 → 0 → ⇒ {-1, 5, 3, 4, 0}

② {-1, 5, 3, 4, 0} ⇒ {-1, 0, 3, 4, 5}

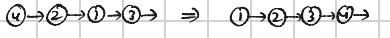
③ {-1, 0, 3, 4, 5} ⇒ -1 → 0 → 3 → 4 → 5 →

(4)

148 Sort List (Medium) (Arrays, Linked List, Sorting) ← Same to the previous Question

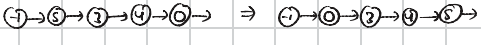
Given the head of a linked list, return the list after sorting it in ascending order.

Example 1:-



head = [4,2,1,3] Output = [1,2,3,4]

Example 2:-



head = [-1,5,3,4,0] Output = [-1,0,3,4,5]

Code:-

```
public int[] linkedListToArray(ListNode head){
    int size = 0;
    ListNode curr = head;

    while(curr != null){
        size++;
        curr = curr.next;
    }

    int[] arr = new int[size];
    curr = head;
    int i = 0;

    while(curr != null){
        arr[i] = curr.val;
        i++;
        curr.next;
    }
    return arr;
}

public ListNode sortList(ListNode head){
    int[] arr = linkedListToArray(head);
    insertSort(arr);
    return arrayToLinkedList(arr);
}
```

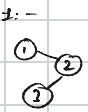
```
public ListNode arrayToLinkedList(int[] arr){

    if(arr == null || arr.length == 0) return null;

    ListNode head = new ListNode(arr[0]);
    ListNode curr = head;

    for(int i = 1; i < arr.length; i++){
        curr.next = new ListNode(arr[i]);
        curr = curr.next;
    }
    return head;
}

public void insertSort(int[] arr){
    if(arr == null || arr.length == 1) return;
    for(int i = 1; i < arr.length; i++){
        int key = arr[i];
        int j = i - 1;

        while(j >= 0 && arr[j] > key){
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
}
```

5

94. Binary Tree Inorder Traversal (Easy) (Trees, Stacks)

Given the root of a binary tree, return the inorder traversal of its nodes' values.

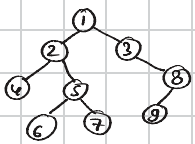
Eg 1:-



root = [1, null, 2, 3]

output = [1, 3, 2]

Eg 2:-



root = [1, 2, 3, 4, 5, null, 8, null, null, 6, 7, 9]

output = [4, 2, 6, 5, 7, 1, 3, 9, 8]

Eg 3:-

root = [] output = []

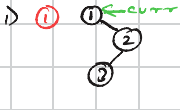
Eg 4:-

root = [1] output = [1]

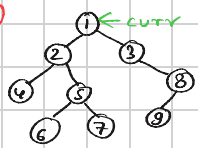
Code:-

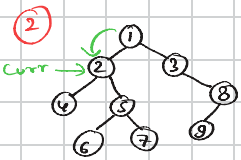
```
public List<Integer> inorderTraversal(TreeNode root){
    List<Integer> result = new LinkedList<>();

    if(root == null){
        return result;
    }

    Stack<TreeNode> st = new Stack<>();
    TreeNode curr = root;

    while(curr != null || !st.isEmpty()){
        while(curr != null){
            st.push(curr);
            curr = curr.left;
        }

        curr = st.pop();
        result.add(curr.val);

        curr = curr.right;
    }

    return result;
}
```

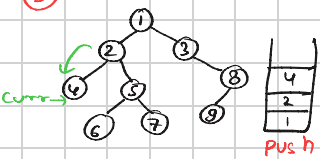
1)

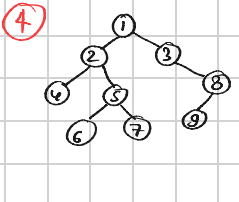


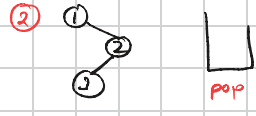
2)

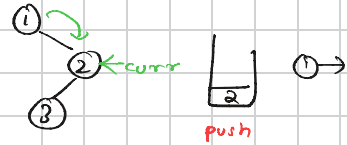


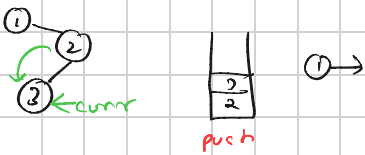


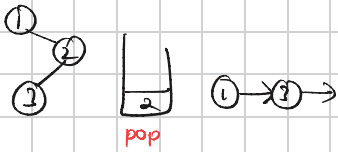


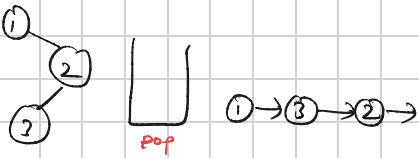




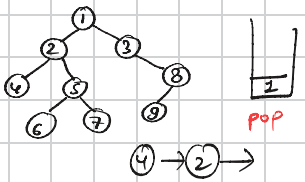




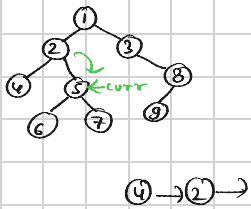




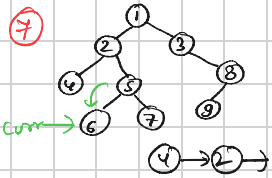
5



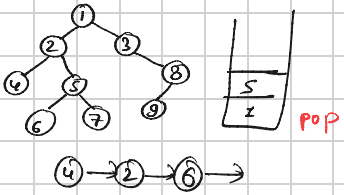
6



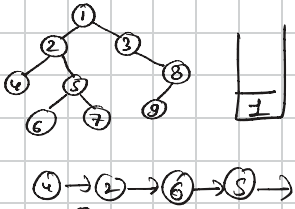
7



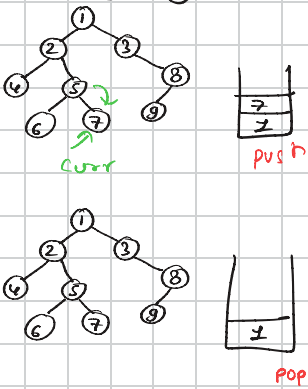
8



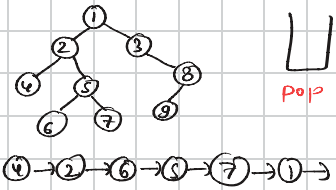
9



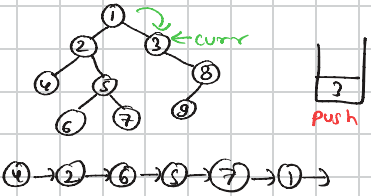
10



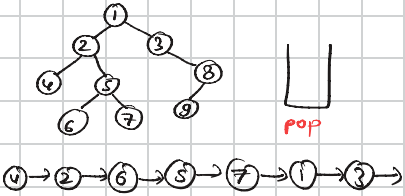
11



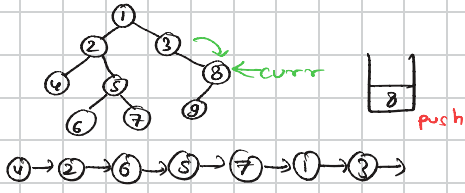
12



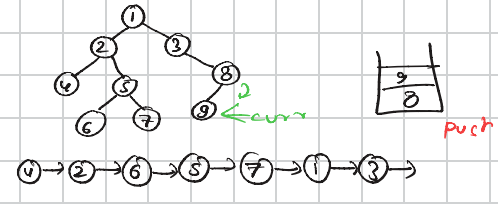
13



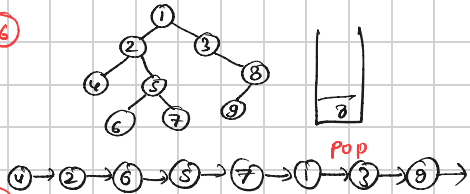
14



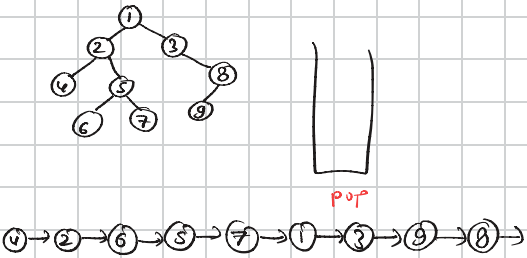
15



16



17

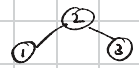


6

98. Validate Binary Search Tree (Medium) (Trees, Stacks)

Given the root of a binary tree, determine if it is a valid binary search tree (BST).
A valid BST is defined as follows:
The left subtree of a node contains only nodes with keys strictly less than the node's key.
The right subtree of a node contains only nodes with keys strictly greater than the node's key.
Both the left and right subtrees must also be binary search trees.

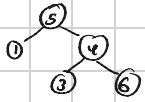
Eg1:-



root = [2,1,3]

Output = true

Eg2:-



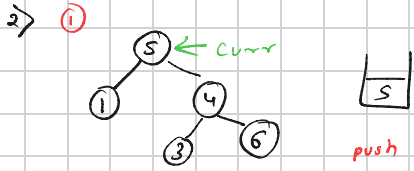
root = [5,1,4,null,null,3,6]

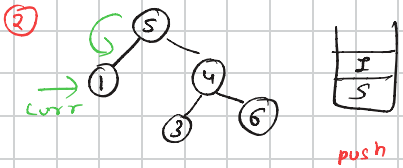
Output = false

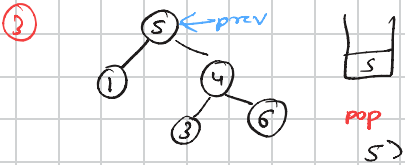
Code:-

```
public static boolean isValidBST(TreeNode root) {
    if (root == null) {
        return true;
    }

    Stack<TreeNode> st = new Stack<>();
    TreeNode curr = root;
    TreeNode pre = null;

    while(curr != null || !st.isEmpty()){
        // Push onto the stack the left nodes to reach the left most node
        while(curr != null){
            st.push(curr);
            curr = curr.left;
        }

        // Capturing the root node ( Left Root Right )
        curr = st.pop();

        // Comparing the previous node with the current node
        if(pre != null && curr.val <= pre.val){
            return false;
        }

        pre = curr;

        // Traverse to the right nodes
        curr = curr.right;
    }

    return true;
}
```

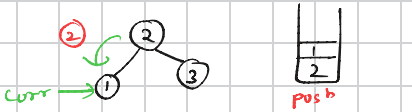
2>

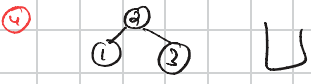






1>





Total true

100. Same Tree (Easy) (Trees)

Given the roots of two binary trees p and q, write a function to check if they are the same or not. Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Eg1:-

p = [1,2,3] q = [1,2,3]

Output = true

Eg2:-

p = [1,2] q = [1,null,2]

Output = false

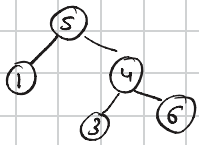
Eg3:-

p = [1,2,1] q = [1,1,2]

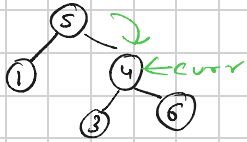
Output = false

Code:-

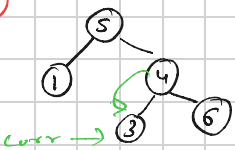
```
public static boolean isSameTree(TreeNode p, TreeNode q) {
    if(p == null && q == null) return true;

    if(p == null || q == null) return false;

    if(p.val == q.val && ( isSameTree(p.left,q.left) == true &&
isSameTree(p.right, q.right) == true )){
        return true;
    }else{
        return false;
    }
}
```



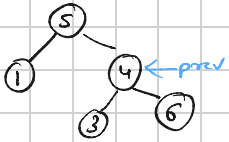
pop



push



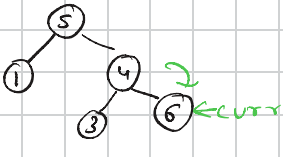
push



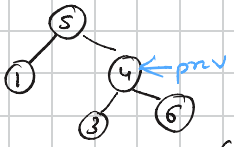
pop

4 > 3

T



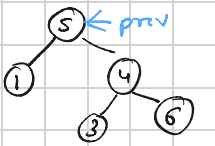
push



6 > 4

T

pop

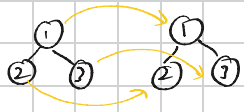


4 > 5

F

pop

1)



1=1 2=2 3=3

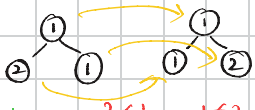
True

2)

1=1 2≠ null null≠2

False

3)



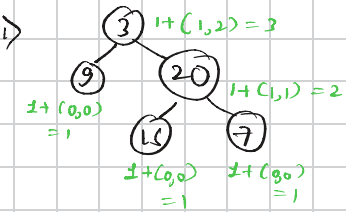
1 = 1 2 ≠ 1 1 ≠ 2

Fals

Code:-

```
public int maxDepth(TreeNode root) {
    if(root == null) return 0;

    return 1 +
Math.max(maxDepth(root.left),
maxDepth(root.right));
  }
```

1)

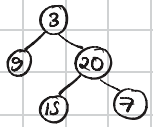


8

104. Maximum Depth of Binary Tree (Easy) (Trees)

Given the root of a binary tree, return its maximum depth. A binary tree's maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

Eg1:-

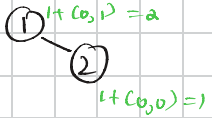


root = [3,9,20,null,null,15,7]

Output = 3

Eg2:- root = [1,null,2] Output = 2

2)



9

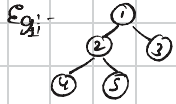
543. Diameter of Binary Tree (Easy) (Trees)

Given the root of a binary tree, return the length of the diameter of the tree. The diameter of a binary tree is the length of the longest path between any two nodes in a tree. This path may or may not pass through the root. The length of a path between two nodes is represented by the number of edges between them.

Code:-

```
int diameter = 0;

    public int diameterOfBinaryTree(TreeNode root) {
        maxDepth(root);
        return diameter;
    }

    public int maxDepth(TreeNode root) {
        if (root == null) return 0;

        int maxDepthLeft = maxDepth(root.left);
        int maxDepthRight = maxDepth(root.right);

        diameter = Math.max(diameter, maxDepthLeft + maxDepthRight);

        return 1 + Math.max(maxDepthLeft, maxDepthRight);
    }
```

Eg1:-

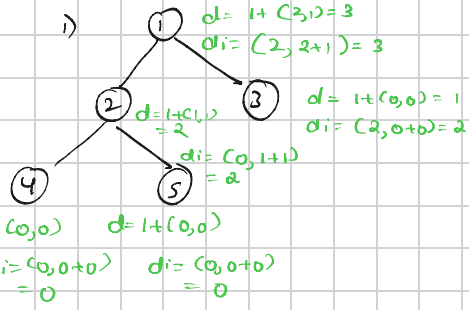


Eg2:- root = [1,2]

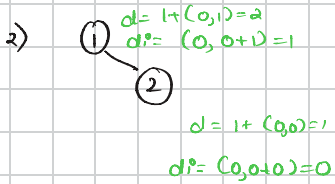
root = [1,2,3,4,5]

Output = 3

1)



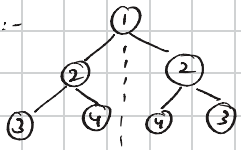
2)



101. Symmetric Tree (Easy) (Trees)

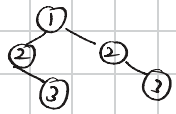
Given the root of a binary tree, check whether it is a mirror of itself (i.e., symmetric around its center).

Eg1:-



root = [1,2,2,3,4,4,3]
Output = true

Eg2:-

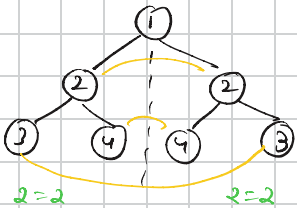


root = [1,2,2,null,3,null,3]
Output = false

Code:-

```
public boolean isSymmetric(TreeNode root) {

    if(root == null) return true;

    return isMirrorTree(root.left,root.right) && isMirrorTree(root.right,root.left);

}

public static boolean isMirrorTree(TreeNode p, TreeNode q) {
    if(p == null && q == null) return true;

    if(p == null || q == null) return false;

    if(p.val == q.val && ( isMirrorTree(p.left,q.right) == true && isMirrorTree(p.right, q.left) == true )){
        return true;
    }else{
        return false;
    }
}
```

1)

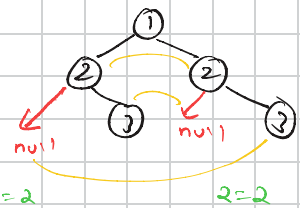


2=2 2=2

3=3 4=4 4=4 3=3

T T

2)



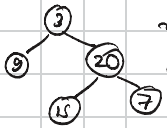
2=2 2=2

null≠3 3≠null null≠3 3≠null

110. Balanced Binary Tree (Easy) (Trees)

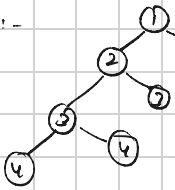
Given a binary tree, determine if it is height-balanced.

Eg1:-



root = [3,9,20,null,null,15,7]
Output = true

Eg2:-

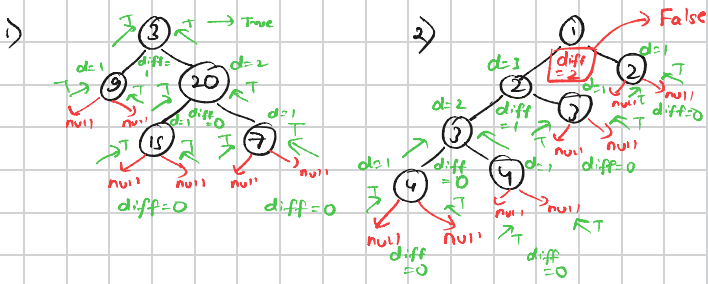


root = [1,2,2,3,3,null,null,4,4]
Output = false

Eg3:- root = [] output = false

Code:-

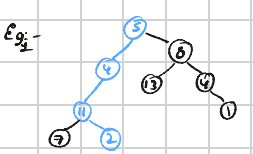
```
public int maxDepth(TreeNode root) {
    if(root == null) return 0;

    int maxDepthLeft = maxDepth(root.left);
    int maxDepthRight = maxDepth(root.right);

    return 1 + Math.max(maxDepthLeft, maxDepthRight);
}

public boolean isBalanced(TreeNode root) {
    if(root == null) return true;

    int diff = Math.abs(maxDepth(root.left) - maxDepth(root.right));

    if(diff <= 1 &&  (isBalanced(root.left) == true && isBalanced(root.right) == true)){
        return true;
    }else{
        return false;
    }
}
```



12

112. Path Sum (Easy) (Trees)

Given the root of a binary tree and an integer targetSum, return true if the tree has a root-to-leaf path such that adding up all the values along the path equals targetSum. A leaf is a node with no children.



Eg1:-

root = [5,4,8,11,null,13,4,7,2,null,null,1]

targetSum = 22

Output = true

Eg2:- root = [1,2,3] targetSum = 5

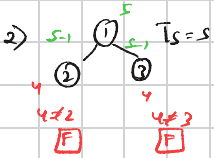
Output = false

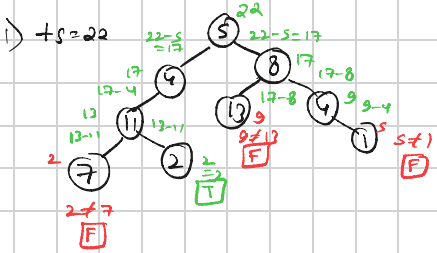
Eg3:- root = [] targetSum = 0

Output = false

Code:-

```
public boolean hasPathSum(TreeNode root, int targetSum) {

    if(root == null) return false;

    if(root.left == null && root.right == null) return targetSum == root.val;

    return hasPathSum(root.left , targetSum - root.val) || hasPathSum(root.right, targetSum - root.val);
  }
```

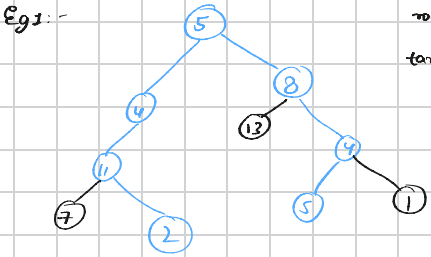




13

113. Path Sum -II (Medium) (Trees)

Given the root of a binary tree and an integer targetSum, return all root-to-leaf paths where the sum of the node values in the path equals targetSum. Each path should be returned as a list of the node values, not node references. A root-to-leaf path is a path starting from the root and ending at any leaf node. A leaf is a node with no children.



Eg1:-

root = [5,4,8,11,null,13,4,7,2,null,null,5,1]

targetSum = 22

Eg2:-

root = [1,2,3]

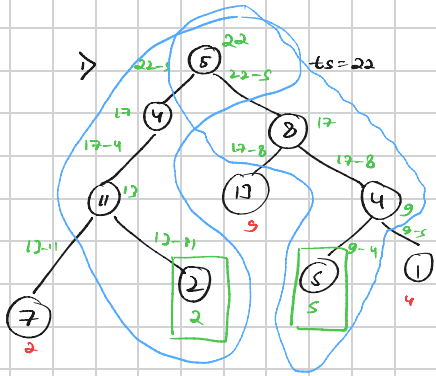
targetSum = 5

Eg2:- root = [1,2]

targetSum = 0

Code :-

```
public List<List<Integer>> pathSum(TreeNode root, int targetSum) {
        List<List<Integer>> result = new ArrayList();
        List<Integer> solution = new ArrayList<Integer>();
        pathSum(root, targetSum, solution, result);
        return result;

    }

    void pathSum(TreeNode root, int sum, List<Integer> solution, List<List<Integer>> result){
        if(root == null){
            return;
        }

        solution.add(root.val);

        if(root.left == null && root.right == null && sum == root.val){
            result.add(new ArrayList<Integer>(solution));
        }else{
            pathSum(root.left,sum - root.val, solution,result);
            pathSum(root.right, sum - root.val, solution, result);
        }

        solution.remove(solution.size() - 1);
    }
```



(1) for path one left subtree

a) solution = {5} add

b) solution = {5, 4} add

c) solution = {5, 4, 11} add

d) left subtree → solution = {5, 4, 11, 7} add → remove (7) then add

c) right subtree → solution = {5, 4, 11, 2} = 22

result = {{5, 4, 11, 2}} add

(2) for path two right subtree

a) solution = {5} add

b) solution = {5, 8} add

left st / right st

c) solution = {5, 8, 13} add → remove (13) then add

d) solution = {5, 8, 4}

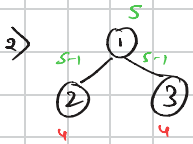
left st / right st

e) solution = {5, 8, 4, 5} = 22 → remove (5) then add

f) solution = {5, 8, 4, }

result = {{5, 4, 11, 2}, {5, 8, 4, 5}}

2)



(1) left

solution = {1} add

solution = {1, 2} add

(2) right

solution = {1} added

solution = {1, 3} added

No sol^n

result = { }

20. Valid Paren the ses (Easy) (Stacks)

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.
An input string is valid if:
Open brackets must be closed by the same type of brackets.
Open brackets must be closed in the correct order.
Every close bracket has a corresponding open bracket of the same type.

Example 1:
Input: s = "()"
Output: true

Example 2:
Input: s = "()[]{}"
Output: true

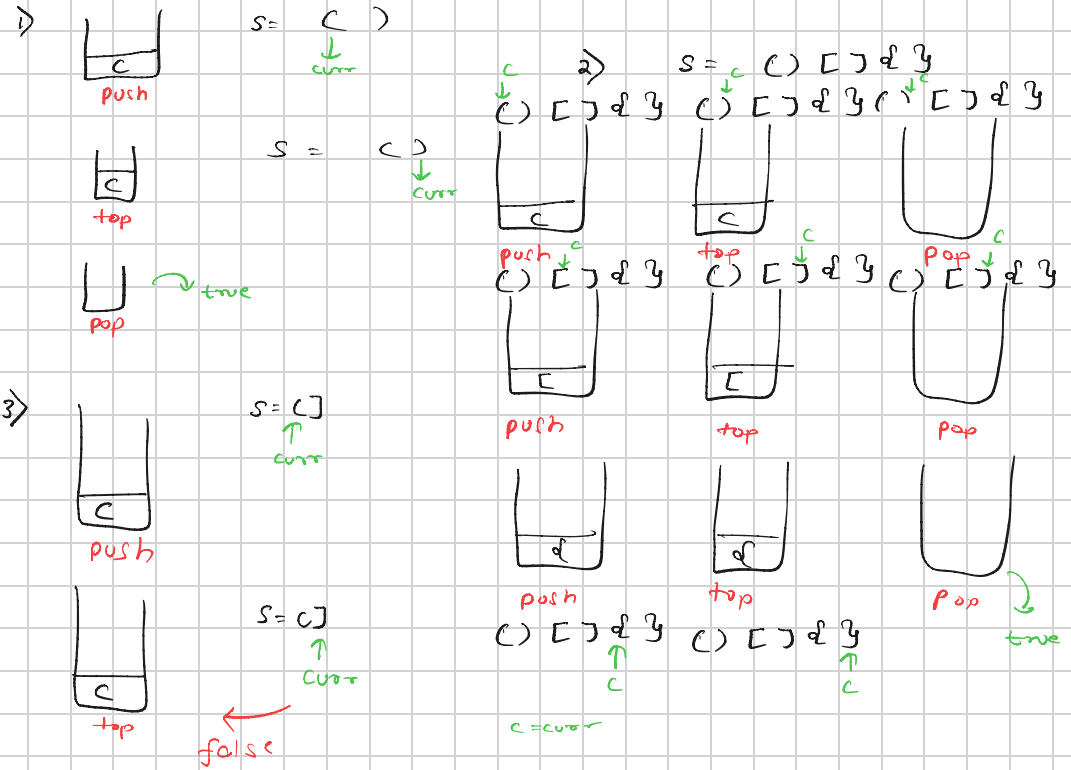
Example 3:
Input: s = "(]"
Output: false

Example 4:
Input: s = "([])"
Output: true

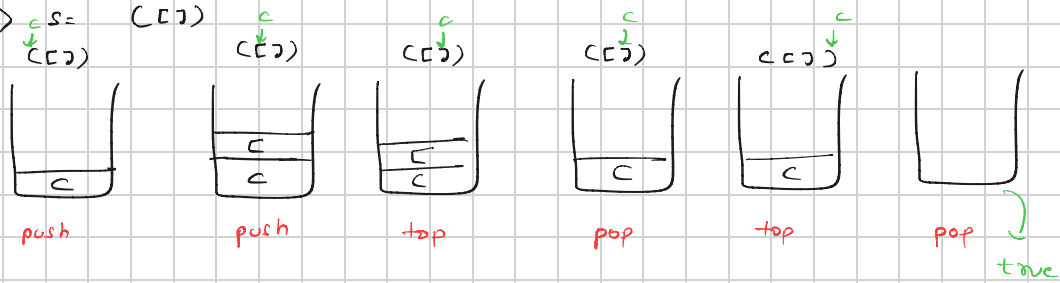
Example 5:
Input: s = "([)]"
Output: false

Code :-

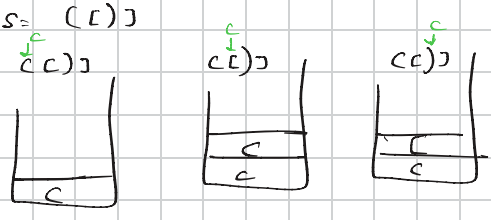
```
public static boolean isValid(String s) {

    Stack<Character> stack = new Stack<>();

    for (int i = 0; i < s.length(); i++) {
        char curr = s.charAt(i);

        if (curr == '(' || curr == '[' || curr == '{') {
            stack.push(curr);
        }
        else if (stack.isEmpty()) {
            return false;
        }
        else {
            Character top = stack.peek();

            if ((top == '[' && curr == ']') ||
                    (top == '{' && curr == '}') ||
                    (top == '(' && curr == ')')) {

                stack.pop();

            } else {
                return false;
            }
        }
    }

    return stack.isEmpty();
}
```



4) S = ([ ])



5) S = ([ )]



15

946. Valid Stack Sequences (Medium) (Stacks)

Given two integer arrays pushed and popped each with distinct values, return true if this could have been the result of a sequence of push and pop operations on an initially empty stack, or false otherwise.

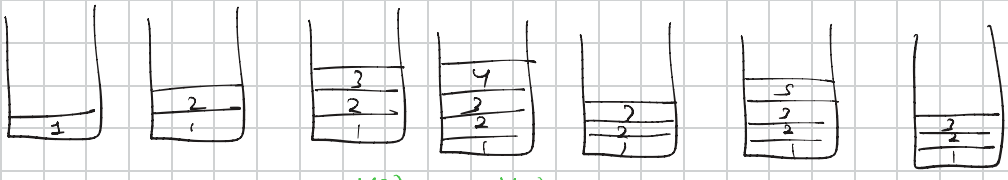
Example 1:
Input: pushed = [1,2,3,4,5], popped = [4,5,3,2,1]
Output: true
Explanation: We might do the following sequence:
push(1), push(2), push(3), push(4),
pop() -> 4,
push(5),
pop() -> 5, pop() -> 3, pop() -> 2, pop() -> 1

Example 2:
Input: pushed = [1,2,3,4,5], popped = [4,3,5,1,2]
Output: false
Explanation: 1 cannot be popped before 2.

Code:-

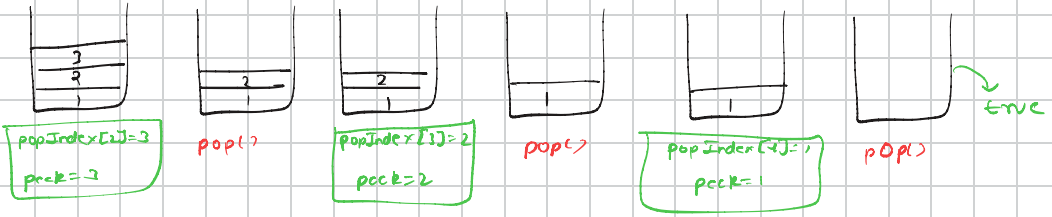
```
public boolean validateStackSequences(int[] pushed, int[] popped) {

    if(pushed.length == 1) return pushed[0] == popped[0];
    Stack<Integer> st = new Stack<>();
    int popIndex = 0;



    for(int ele : pushed){
        st.push(ele);

        while(!st.isEmpty() && st.peek() == popped[popIndex]){
            st.pop();
            popIndex++;
        }
    }

    return st.isEmpty();

}
```

1) pushed = [1,2,3,4,5] popped = [4,5,3,2,1]

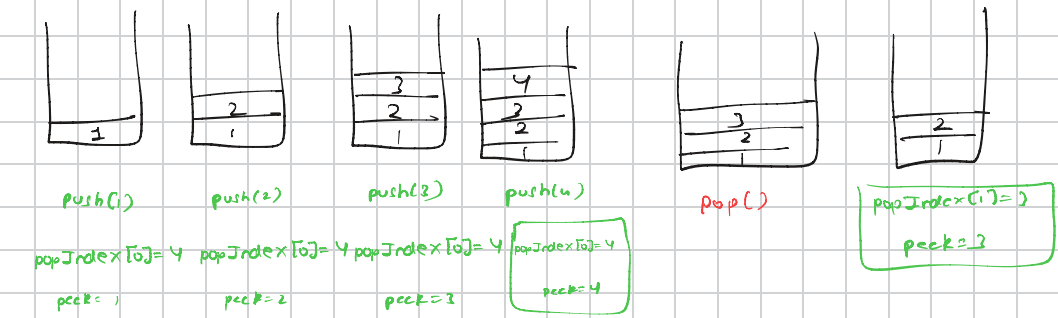


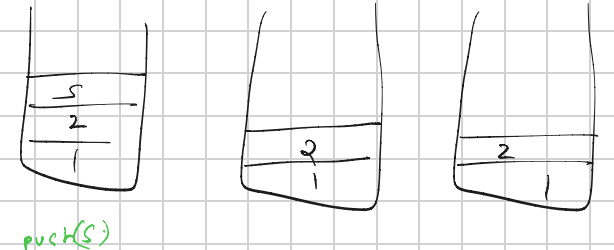
popIndex[0] = 4 popIndex[0] = 4 popIndex[0] = 4 popIndex[0] = 4 popIndex[1] = 5

peek = 1 peek = 2 peek = 3 peek = 4 peek = 5



1) pushed = [1,2,3,4,5] popped = [4,3,5,1,2]





16

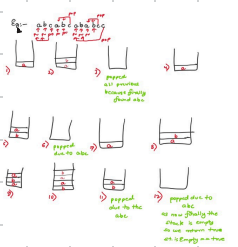
1003. Check If Word is Valid After Substitution (Medium) (Stacks)

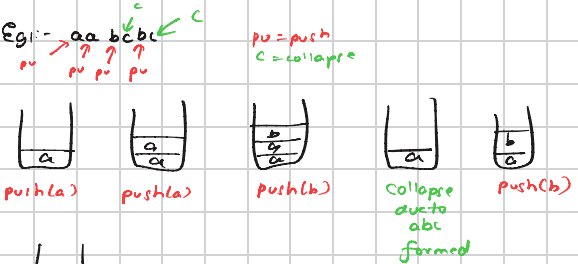
Given a string s, determine if it is valid. A string s is valid if, starting with an empty string t = "", you can transform t into s after performing the following operation any number of times: Insert string "abc" into any position in t. More formally, t becomes tleft + "abc" + tright, where t == tleft + tright. Note that tleft and tright may be empty. Return true if s is a valid string, otherwise, return false.

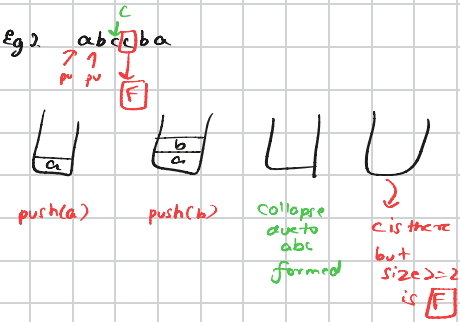
Example 1:
Input: s = "aabcbc"
Output: true
Explanation:
"" -> "abc" -> "aabcbc"
Thus, "aabcbc" is valid.
Example 2:
Input: s = "abcabcababcc"
Output: true
Explanation:
"" -> "abc" -> "abcabc" -> "abcabcabc" -> "abcabcababcc"
Thus, "abcabcababcc" is valid.
Example 3:
Input: s = "abccba"
Output: false
Explanation: It is impossible to get "abccba" using the operation.

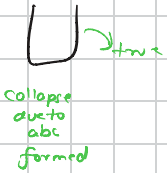
Code:-

```
public boolean isValid(String s) {
        if (s.length() % 3 != 0) {
            return false;
        }

        Stack<Character> st = new Stack<>();

        //Enhanced for - loop
        for(char ch: s.toCharArray()) {
            if(ch == 'c') {
                if(st.size() >=2 && st.pop()=='b' && st.pop()=='a') {
                    continue; //do nothing;
                } else {
                    return false;
                }
            } else {
                //Push for a and b elements
                st.push(ch);
            }
        }
        return st.isEmpty();
    }
```









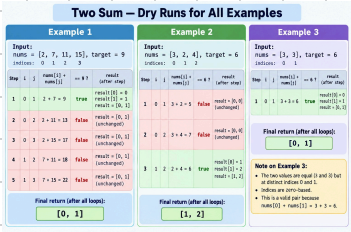
1. Two Sum (Easy) (Arrays)

You are given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.
You may assume that each input would have exactly one solution, and you may not use the same element twice.
You can return the answer in any order.

Example 1;
Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].
Example 2:
Input: nums = [3,2,4], target = 6
Output: [1,2]
Example 3;
Input: nums = [3,3], target = 6
Output: [0,1]

Code:-

```
public int[] twoSum(int[] nums, int target) {
        int[] result = new int[2];

        for(int i = 0; i < nums.length-1 ; i++){
            for(int j = i+1 ; j < nums.length; j++){
                if(nums[i] + nums[j] == target){
                    result[0] = i;
                    result[1] = j;
                }
            }
        }
        return result;
    }
```

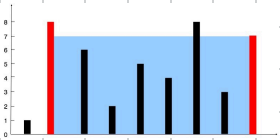


18

11. Container with Most Water (Medium) (Arrays, Two pointers)

You are given an integer array height of length n. There are n vertical lines drawn such that the two endpoints of the ith line are (i, 0) and (i, height[i]). Find two lines that together with the x-axis form a container, such that the container contains the most water. Return the maximum amount of water a container can store. Notice that you may not slant the container.

Example 1:
Input: height = [1,8,6,2,5,4,8,3,7]
Output: 49
Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.

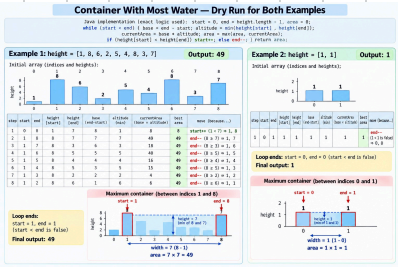


Example 2:

Input: height = [1,1]
Output: 1

Code: -

```
public int maxArea(int[] height) {
    int start = 0;
    int end = height.length - 1;
    int area = 0;

    while(start < end){
        int base = end-start;
        int altitude = Math.min(height[start],height[end]);

        int currentArea = base*altitude;
        area = Math.max(area, currentArea);

        if(height[start] < height[end]){
            start++;
        }else{
            end--;
        }
    }

    return area;
}
```



19

26. Remove Duplicates from Sorted Array (Easy) (Arrays, Two pointers)

Given an integer array nums sorted in non-decreasing order, remove the duplicates in-place such that each unique element appears only once. The relative order of the elements should be kept the same. Consider the number of unique elements in nums to be k. After removing duplicates, return the number of unique elements k. The first k elements of nums should contain the unique numbers in sorted order. The remaining elements beyond index k - 1 can be ignored.

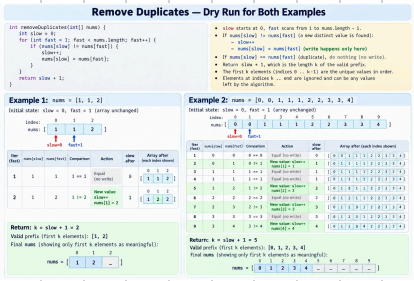
Example 1:

Input: nums = [1,1,2]
Output: 2, nums = [1,2,_]
Explanation: Your function should return k = 2, with the first two elements of nums being 1 and 2 respectively.
It does not matter what you leave beyond the returned k (hence they are underscores).
Example 2:

Input: nums = [0,0,1,1,1,2,2,3,3,4]
Output: 5, nums = [0,1,2,3,4,_,_,_,_,_]
Explanation: Your function should return k = 5, with the first five elements of nums being 0, 1, 2, 3, and 4 respectively.
It does not matter what you leave beyond the returned k (hence they are underscores).

Code: -

```
public int removeDuplicates(int[] nums) {
    int slow = 0;

    for (int fast = 1; fast < nums.length; fast++) {
        if (nums[slow] != nums[fast]) {
            slow++;
            nums[slow] = nums[fast];
        }
    }

    return slow + 1;
}
```



20

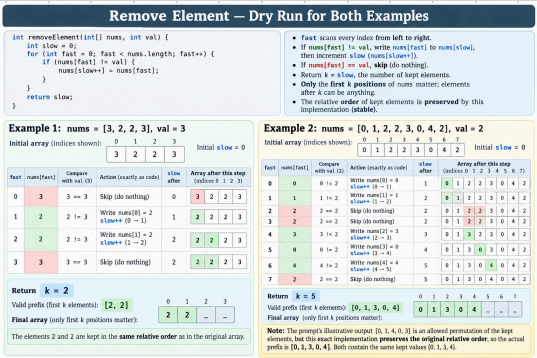
27. Remove Element (Easy) (Arrays)

Given an integer array nums and an integer val, remove all occurrences of val in nums in-place. The order of the elements may be changed. Then return the number of elements in nums which are not equal to val. Consider the number of elements in nums which are not equal to val be k, to get accepted, you need to do the following things: Change the array nums such that the first k elements of nums contain the elements which are not equal to val. The remaining elements of nums are not important as well as the size of nums. Return k.

Code: -

```
public int removeElement(int[] nums, int val) {
    int slow = 0;

    for(int fast = 0; fast < nums.length; fast++){
        if(nums[fast] != val){
            nums[slow++] = nums[fast];
        }
    }

    return slow;

}
```

Example 1:
Input: nums = [3,2,2,3], val = 3
Output: 2, nums = [2,2,_,_]
Explanation: Your function should return k = 2, with the first two elements of nums being 2.
It does not matter what you leave beyond the returned k (hence they are underscores).
Example 2:
Input: nums = [0,1,2,2,3,0,4,2], val = 2
Output: 5, nums = [0,1,4,0,3,_,_,_]
Explanation: Your function should return k = 5, with the first five elements of nums containing 0, 0, 1, 3, and 4.
Note that the five elements can be returned in any order.
It does not matter what you leave beyond the returned k (hence they are underscores).

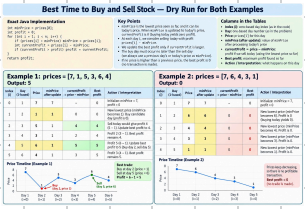


21

121. Best Time to Buy and Sell Stock (Easy) (Arrays)

You are given an array prices where prices[i] is the price of a given stock on the ith day. You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock.Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Example 1:
Input: prices = [7,1,5,3,6,4]
Output: 5
Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.
Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.
Example 2:
Input: prices = [7,6,4,3,1]
Output: 0
Explanation: In this case, no transactions are done and the max profit = 0.



Code:-

```
public int maxProfit(int[] prices) {

    int minPrice = prices[0];
    int profit = 0;

    for(int i = 1; i < prices.length; i++) {

        if(prices[i] < minPrice) {
            minPrice = prices[i];
        }

        int currentProfit = prices[i] - minPrice;

        if(currentProfit > profit) {
            profit = currentProfit;
        }
    }

    return profit;
}
```

22

122. Best Time to Buy and Sell Stock - II (Medium) (Arrays)

You are given an integer array prices where prices[i] is the price of a given stock on the ith day.

On each day, you may decide to buy and/or sell the stock. You can only hold at most one share of the stock at any time. However, you can sell and buy the stock multiple times on the same day, ensuring you never hold more than one share of the stock.

Find and return the maximum profit you can achieve.

Example 1:

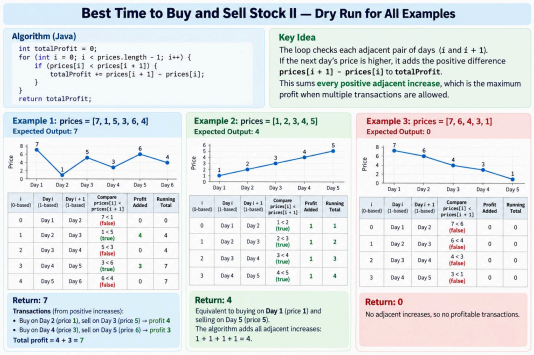
Input: prices = [7,1,5,3,6,4]
Output: 7
Explanation: Buy on day 2 (price = 1) and sell on day 3 (price = 5), profit = 5-1 = 4.
Then buy on day 4 (price = 3) and sell on day 5 (price = 6), profit = 6-3 = 3.
Total profit is 4 + 3 = 7.
Example 2:

Input: prices = [1,2,3,4,5]
Output: 4
Explanation: Buy on day 1 (price = 1) and sell on day 5 (price = 5), profit = 5-1 = 4.
Total profit is 4.
Example 3:

Input: prices = [7,6,4,3,1]
Output: 0
Explanation: There is no way to make a positive profit, so we never buy the stock to achieve the maximum profit of 0.

Code:-

```
public int maxProfit(int[] prices) {
    int totalProfit = 0;

    for(int i = 0; i < prices.length-1; i++){
        if(prices[i] < prices[i+1])
totalProfit += prices[i+1] - prices[i];
    }

    return totalProfit;

}
```



23

189. Rotate Array (Medium) (Arrays, Swapping Algorithm)

Given an integer array nums, rotate the array to the right by k steps, where k is non-negative.

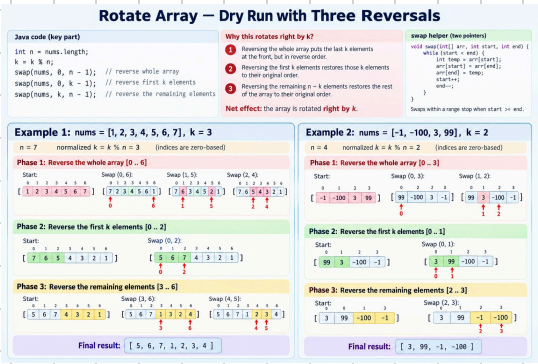
Example 1:

Input: nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
Explanation:
rotate 1 steps to the right: [7,1,2,3,4,5,6]
rotate 2 steps to the right: [6,7,1,2,3,4,5]
rotate 3 steps to the right: [5,6,7,1,2,3,4]
Example 2:

Input: nums = [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]
Explanation:
rotate 1 steps to the right: [99,-1,-100,3]
rotate 2 steps to the right: [3,99,-1,-100]

Code:-

```
public void swap(int[] arr,int start,int end){
    while(start < end){
        int temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
        start++;
        end--;
    }
}
public void rotate(int[] nums, int k) {
    int n = nums.length;
    k = k%n;

    swap(nums,0,n-1);
    swap(nums,0,k-1);
    swap(nums,k,n-1);
}
```



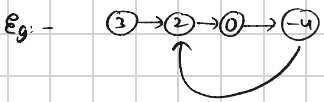
24

141. Linked List Cycle (Easy) (Linked List, Two pointers)

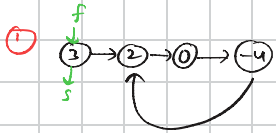
Given head, the head of a linked list, determine if the linked list has a cycle in it. There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter. Return true if there is a cycle in the linked list. Otherwise, return false.

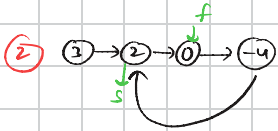
Code :-

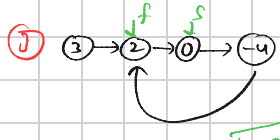
```
public boolean hasCycle(ListNode head)
{
    if(head == null || head.next == null){
        return false;
    }

    ListNode slow = head;
    ListNode fast = head;

    while(fast!= null && fast.next != null){
        fast = fast.next.next;
        slow = slow.next;

        if(slow == fast){
            return true;
        }
    }

    return false;
}
```

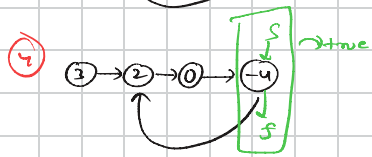
Eg :- 

head = [3, 2, 0, -4]
pos = 1 Output = true

i) 

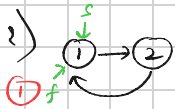
2 

3 

4 

Eg :-

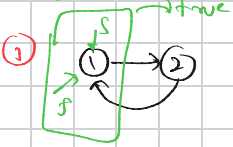
head = [1, 2]
pos = 0 Output = true

i) 

Eg2) :- head [ ] pos = -1
Output = false

1

2

1 

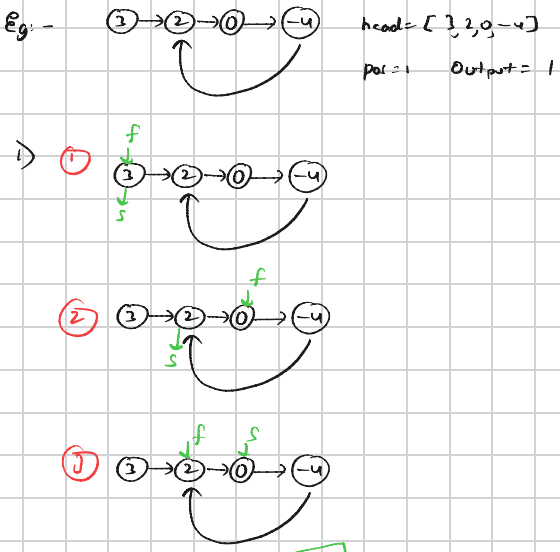
25

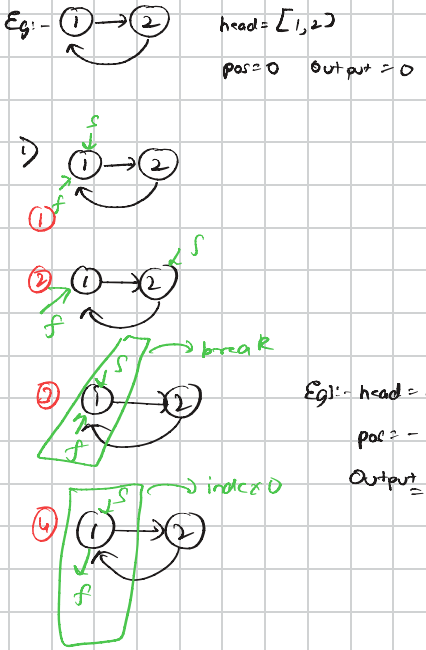
142. Linked List Cycle - II (Medium) (Linked List, Two Pointer)

Given the head of a linked list, return the node where the cycle begins. If there is no cycle, return null. There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that tail's next pointer is connected to (0-indexed). It is -1 if there is no cycle. Note that pos is not passed as a parameter. Do not modify the linked list.

Code :-

```
public ListNode detectCycle(ListNode head) {
    if(head == null || head.next == null) return null;

    ListNode slow = head , fast = head;

    while(fast != null && fast.next != null){
        slow = slow.next;
        fast = fast.next.next;

        if(slow == fast) break;
    }

    if(fast == null || fast.next == null) return null;

    slow = head;

    while(slow != fast){
        slow = slow.next;
        fast = fast.next;
    }

    return slow;

}
```





Eg:- head = [1]

pos = -1

Output = no cycle
so
no index